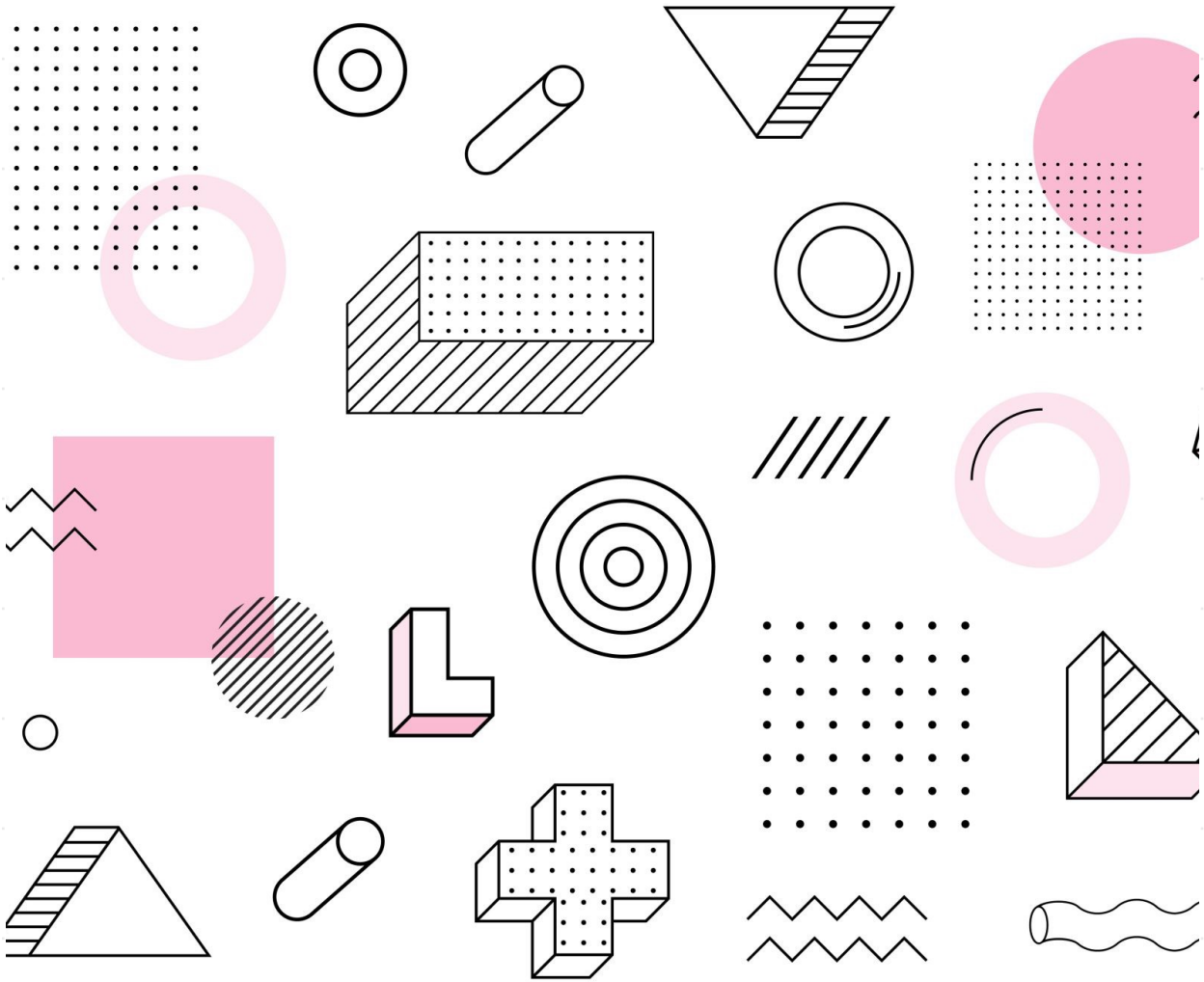
Chapter 13: Code Generation

陳奇業 成功大學資訊工程系



Generating Instructions from Three-address Code

Example: D = (A*B)+C

	0	1	2
=*	A	B	T1
=+	T1	C	T2
=	T2		D

Skeletons

將quatoduad拉成指令

```
=+  =>   Load  R1  0 (first parameter)
         Add   R1  1 (2nd parameter)
         Stor  R1  2 (3rd parameter)

=*  =>   Load  R1  0 (first parameter)
         Mul   R1  1 (2nd parameter)
         Stor  R1  2 (3rd parameter)

=   =>   Load  R1  0 (first parameter)
         Stor  R1  2 (3rd parameter)
```

Thus we get:

```
Load  R1  A
Mul   R1  B
Stor  R1  T1
Load  R1  T1
Add   R1  C
Stor  R1  T2
Load  R1  T2
Stor  R1  D
```

There are 8 instructions.

Additional skeleton (pseudo-operators)

- FX : fetch if not already available (有需要才fetch)
- **Code generator must remember what is in each register.**

Thus we have

```
=+  =>  FX    R1  0 (first parameter)
        Add   R1  1 (2nd parameter)
        Stor  R1  2 (3rd parameter)

=*  =>  FX    R1  0
        Mul   R1  1
        Stor  R1  2

=   =>  FX    R1  0
        Stor  R1  2
```

Thus we get

```
Load  R1  A
Mul   R1  B
Stor  R1  T1
Load  R1  T1
Add   R1  C
Stor  R1  T2
Load  R1  T2
Stor  R1  D
```

```
Load  R1  A
Mul   R1  B
Stor  R1  T1
Add   R1  C
Stor  R1  T2
Stor  R1  D
```

There are 6 instructions

Another Pseudo Operator SX

- **SX** – store if not used right away (or if it is not a temporary variable (e.g., T1, T2, etc.)) (如果沒有馬上使用才store)
- Thus we have:

```
=+  =>  FX   R1  0 (first parameter)     =*   => FX    R1   0
        Add   R1  1 (2nd parameter)            Mul    R1   1
        SX    R1  2 (3rd parameter)            SX    R1   2

 =   =>  FX    R1  0
         SX    R1  2
```

Thus we finally get:

```
Load  R1  A
Mul   R1  B
Stor  R1  T1
Add   R1  C
Stor  R1  T2
Stor  R1  D
```

```
Load  R1  A
Mul   R1  B
Add   R1  C
Stor  R1  D
```

There are 4 instructions.

Figure 8.15
Complete instruction set of the Tiny Machine

RO Instructions

Format: ***opcode r,s,t***

Opcode	Effect
HALT	stop execution (operands ignored)
IN	**reg[*r*]** ← integer value read from the standard input (***s*** and ***t*** ignored)
OUT	**reg[*r*]** → the standard output (***s*** and ***t*** ignored)
ADD	**reg[*r*] = reg[*s*] + reg[*t*]**
SUB	**reg[*r*] = reg[*s*] - reg[*t*]**
MUL	**reg[*r*] = reg[*s*] * reg[*t*]**
DIV	**reg[*r*] = reg[*s*] / reg[*t*]** (may generate **ZERO_DIV**)

RM Instructions

Format: ***opcode r,d(s)***

(***a*** = ***d*** + **reg[*s*]**; any reference to **dMem[*a*]** generates **DMEM_ERR** if ***a*** < 0 or ***a*** ≥ **DADDR_SIZE**)

Opcode	Effect
LD	**reg[*r*] = dMem[*a*]** (load ***r*** with memory value at ***a***)
LDA	**reg[*r*] = *a*** (load address ***a*** directly into ***r***)
LDC	**reg[*r*] = *d*** (load constant ***d*** directly into ***r***—***s*** is ignored)
ST	**dMem[*a*] = reg[*r*]** (store value in ***r*** to memory location ***a***)
JLT	**if (reg[*r*]<0) reg[PC_REG] = *a*** (jump to instruction ***a*** if ***r*** is negative, similarly for the following)
JLE	**if (reg[*r*]<=0) reg[PC_REG] = *a***
JGE	**if (reg[*r*]>=0) reg[PC_REG] = *a***
JGT	**if (reg[*r*]>0) reg[PC_REG] = *a***
JEQ	**if (reg[*r*]==0) reg[PC_REG] = *a***
JNE	**if (reg[*r*]!=0) reg[PC_REG] = *a***

Figure 8.16

A TM program showing format conventions

```
* This program inputs an integer, computes
* its factorial if it is positive,
* and prints the result

0:     IN    0,0,0        r0 = read
1:     JLE   0,6(7)       if 0 < r0 then   r7=1
2:     LDC   1,1,0          r1 = 1
3:     LDC   2,1,0          r2 = 1
                          * repeat
4:     MUL 1,1,0             r1 = r1 * r0
5:     SUB 0,0,2             r0 = r0 - r2
6:     JNE   0,-3(7)        until r0 == 0  r7=6
7:     OUT   1,0,0          write r1
8:     HALT 0,0,0         halt
* end of program
```